PDS Trust

Software Design Document (SDD) — Competition MVP

Version 1.0 | 27 October 2026 | Status: Proposed design, not implementation evidence

1. Design objective

Build a demonstrable Laravel application for three PDS stakeholders: beneficiaries, Fair Price Shop (FPS) owners and government officials. The defining capability is traceable reconciliation of stock records and beneficiary-verified distribution, with one shared verification service behind web and simulated SMS channels. No real government or telecom integration is claimed.

2. Scope and assumptions

- MVP: authenticated role-based portals, entitlement viewing, allocation/receipt, append-only inventory ledger, distribution, web verification, SMS simulator, discrepancy cases, grievances, audit logs, dashboards and automated tests.
- Seeded system administrator is for development setup only; no separate admin UI.
- Only synthetic beneficiaries, shops, ration-card identifiers and phone numbers are used in demonstration data.
- SMS simulator mimics an inbound/outbound gateway; it cannot send or receive actual cellular messages.
- Future only: IVR, USSD, real SMS gateway, offline sync, ML anomaly detection, official PDS integrations.

3. Architecture

Browser (beneficiary / FPS / official) → Laravel routes + middleware → controllers / form requests → domain services → Eloquent models → MySQL. SMS simulator UI → simulated inbound-message controller → same VerificationService. Domain events trigger notification outbox, discrepancy creation and audit records.

Component	Responsibility
Auth + policies	Session authentication; per-resource authorization and shop ownership checks
AllocationService	Allocate stock and compare FPS receipt with allocation
InventoryService	Append stock movements; calculate balances; reject negative stock
DistributionService	Check entitlement and stock; create distribution and ledger debit atomically
VerificationService	Accept web or validated simulated SMS response; preserve history; update status
DiscrepancyService	Create/idempotently link stock and beneficiary mismatch cases
SmsSimulatorService	Render outbound messages, parse inbound commands and persist processing outcomes
AuditService	Append actor, action, target, timestamp and safe metadata

4. Domain model and relational schema

Suggested tables and essential fields. Each table also has id, created_at and updated_at unless append-only or otherwise specified. Exact migration names may be adjusted without changing the relationships.

Table	Key columns / constraints
users	name, email UNIQUE, password_hash, role ENUM(beneficiary,fps_owner,official,admin), active
beneficiary_profiles	user_id UNIQUE FK, ration_card_mask, phone_e164 UNIQUE nullable, preferred_language, sms_opt_in
fair_price_shops	code UNIQUE, name, location_label, owner_user_id FK, active
commodities	name, unit (kg), active
entitlements	beneficiary_id FK, commodity_id FK, period (YYYY-MM), quantity; UNIQUE beneficiary+commodity+period
stock_allocations	shop_id FK, commodity_id FK, period, quantity, allocated_by FK, status
stock_receipts	allocation_id FK, received_quantity, received_by FK, received_at
inventory_ledger	shop_id FK, commodity_id FK, movement_type, signed_quantity, source_type, source_id, actor_id, created_at; append-only
distribution_transactions	reference UNIQUE, beneficiary_id FK, shop_id FK, commodity_id FK, period, quantity, issued_by FK, issued_at, verification_status
verification_responses	transaction_id FK, channel, response_type, valid, reason_code, actor_or_phone_ref, received_at, request_key UNIQUE nullable
sms_messages	transaction_id FK nullable, direction, recipient_mask, body, simulation_status, processing_result, created_at
discrepancy_cases	case_ref UNIQUE, case_type, source_type, source_id, status, assigned_official_id FK nullable, resolution_note
grievances	beneficiary_id FK, transaction_id FK nullable, description, status, resolution_note
audit_logs	actor_id FK nullable, action, target_type, target_id, safe_metadata JSON, created_at; no ordinary update/delete

Relationship summary: users 1—1 beneficiary_profiles; users 1—many shop ownership; shops 1—many allocations, ledger movements and transactions; transactions 1—many verification_responses and sms_messages; discrepancy_cases polymorphically reference a specific allocation/receipt/distribution event. Enforce FK constraints and indexes on reference, shop_id, beneficiary_id, period and status.

5. Critical workflows and transaction boundaries

5.1 Stock allocation and receipt

Official allocates stock → owner records receipt → receipt generates inventory CREDIT → allocation/receipt difference creates a stock discrepancy case. Validate positive quantities and prohibit receipt totals exceeding permitted rules without explicit review.

5.2 Distribution

Within a database transaction: authorise owner and shop → lock relevant stock/entitlement rows or otherwise serialise balance checks → check remaining entitlement and stock → insert distribution with unique reference and status PENDING → append inventory DEBIT → enqueue simulated notification → commit. A failed check rolls back the whole operation.

5.3 Web and SMS verification

Web: authenticated beneficiary selects their own transaction. SMS: simulator submits registered sender, transaction reference, and supported reply; server validates opt-in, matching transaction, reply format and expiry. Both invoke VerificationService. Save every attempted response, deduplicate requests, then transition status and create dispute case when appropriate. Never infer confirmation from nonresponse.

5.4 Dispute after confirmation

A later valid dispute is preserved as a new verification response and opens a review case; the earlier confirmation remains in history. An official cannot silently erase a response. A case resolution does not rewrite stock ledger history.

6. Verification state machine

Current	Input	Result
PENDING	Valid CONFIRM	CONFIRMED
PENDING	Valid DISPUTE	DISPUTED + case
CONFIRMED	Later DISPUTE	DISPUTED + case; retain confirmation
DISPUTED	Any further response	Retain case and append response; no automatic clearing
Any	Invalid/expired/duplicate	No status change; log outcome

A correction or case closure requires a separate authorised official action; do not overwrite original beneficiary evidence.

7. Simulated SMS protocol

Outbound sample: “PDS Trust: 5 kg wheat recorded. Ref TX1042. Reply TX1042 1 to confirm or TX1042 2 to dispute.” Supported responses are deliberately limited to confirmation and dispute; optional “did not visit” can map to dispute reason. Sender phone must match an opted-in beneficiary record. Use test-only sender numbers, rate limits, expiry windows and opaque references. Simulator should show sent/received records and validation outcomes.

8. Routes and authorisation (illustrative)

Method + route	Access	Purpose
GET /beneficiary/transactions	Beneficiary	Own transactions
POST /beneficiary/transactions/{id}/verify	Beneficiary	Confirm/dispute own transaction
POST /fps/distributions	FPS owner	Issue ration at assigned shop
POST /fps/receipts	FPS owner	Record stock receipt
POST /official/allocations	Official	Allocate stock
GET /official/cases	Official	Review cases
POST /official/cases/{id}/resolve	Official	Resolve case with note
GET /demo/sms	Development demo access	View simulated messages
POST /demo/sms/inbound	Development demo access	Simulate inbound reply

Demo endpoints must be disabled or access-restricted outside the competition demonstration environment. Controllers use FormRequest validation, policies and service methods; no direct balance edits.

9. Security, privacy and resilience

- Laravel CSRF protection, password hashing, rate limiting, session authentication and authorisation policies.
- No sensitive full ration-card numbers or phone numbers in logs, SMS templates or screenshots.
- SMS is lower-assurance than web login; sender and transaction matching do not prove identity. Flag disputes for human review.
- Ledger and audit entries are append-only for normal users; privileged database operators are outside the MVP tamper-resistance guarantee.
- Atomic stock writes, idempotent message processing, safe retries and database constraints prevent double deductions.
- If SMS is unavailable, verification stays PENDING and web workflow remains usable.

10. UI screens

Role	Minimum screens
Beneficiary	Login, entitlement, transactions, confirm/dispute, grievances
FPS owner	Dashboard, receipts, stock balance, issue ration, transaction list
Official	Allocation form, stock discrepancies, dispute queue, case detail and resolution
Demo only	SMS outbox, incoming message form, response outcome

11. Testing and acceptance criteria

Test	Expected result
T01 Allocate and receive	Ledger credit equals receipt; mismatch generates case
T02 Issue ration	Atomic debit and transaction; negative stock rejected
T03 Web confirmation	Transaction confirmed and response recorded
T04 SMS confirmation	Simulated inbound command changes same transaction status
T05 SMS dispute	Case generated and visible to official
T06 Invalid/duplicate reply	No second state transition or ledger change
T07 No SMS response	Status remains PENDING
T08 Cross-role access	Unauthorized access rejected
T09 Confirm then dispute	Both responses retained; dispute case created
T10 Case resolution	Official note and audit entry retained

12. Implementation sequence

1. Laravel scaffold, migrations, seeders and roles → 2. Allocation, receipt and ledger → 3. Distribution with transactional checks → 4. Shared VerificationService and web UI → 5. SMS simulator → 6. Discrepancy/case review → 7. Grievances, audit, tests → 8. Demo data, README, screenshots and 3–5 minute video. Prioritise a complete vertical slice over decorative dashboards.

13. Future extensions — not implemented

IVR: a future voice gateway could accept DTMF confirmation/dispute in Hindi and regional languages, translate the call outcome into the same VerificationService request, and create cases as needed. Architecture: telecom IVR provider → authenticated web hook adapter → VerificationService → existing records. Future work must address consent, language, call costs, shared-phone risks, identity assurance and provider integration. Other future items: real SMS delivery, USSD, offline synchronisation, predictive analytics and official PDS APIs.

14. Traceability and deliverables

FR-01→Auth; FR-03–05→Allocation/Inventory; FR-06–07→Distribution; FR-08–09→SMS Simulator; FR-10–12→Discrepancy/Grievance; FR-13→Audit; FR-16–20→Verification/SMS/notification rules. NFR-08→Privacy; NFR-09→Audit. Deliver repository with .env.example, setup README, migrations, seeders, license, tests, architecture document and demo video. This SDD describes intended behaviour, not proof of implemented functionality.

Appendix A — Technical Diagrams

The following ten diagrams describe the proposed PDS Trust competition MVP. They are design specifications, not evidence of implemented features. SMS is simulated; real SMS gateway and IVR are future extensions.

A.1 System architecture

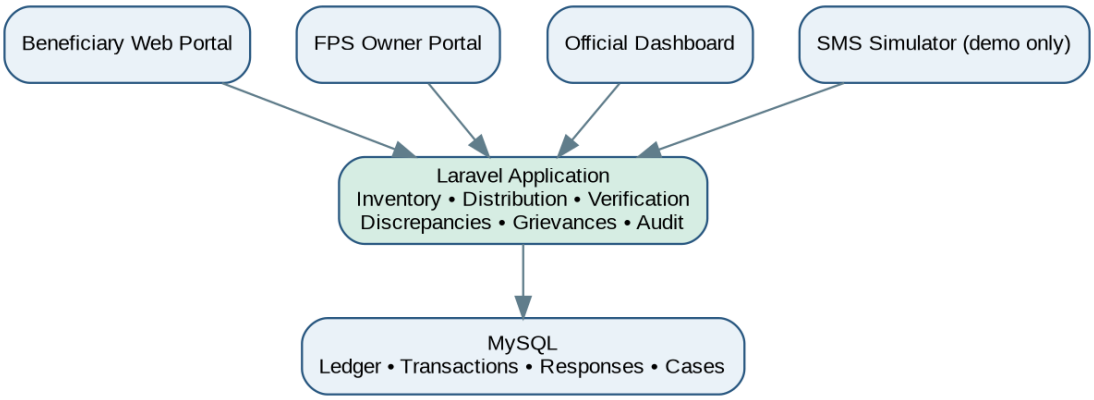
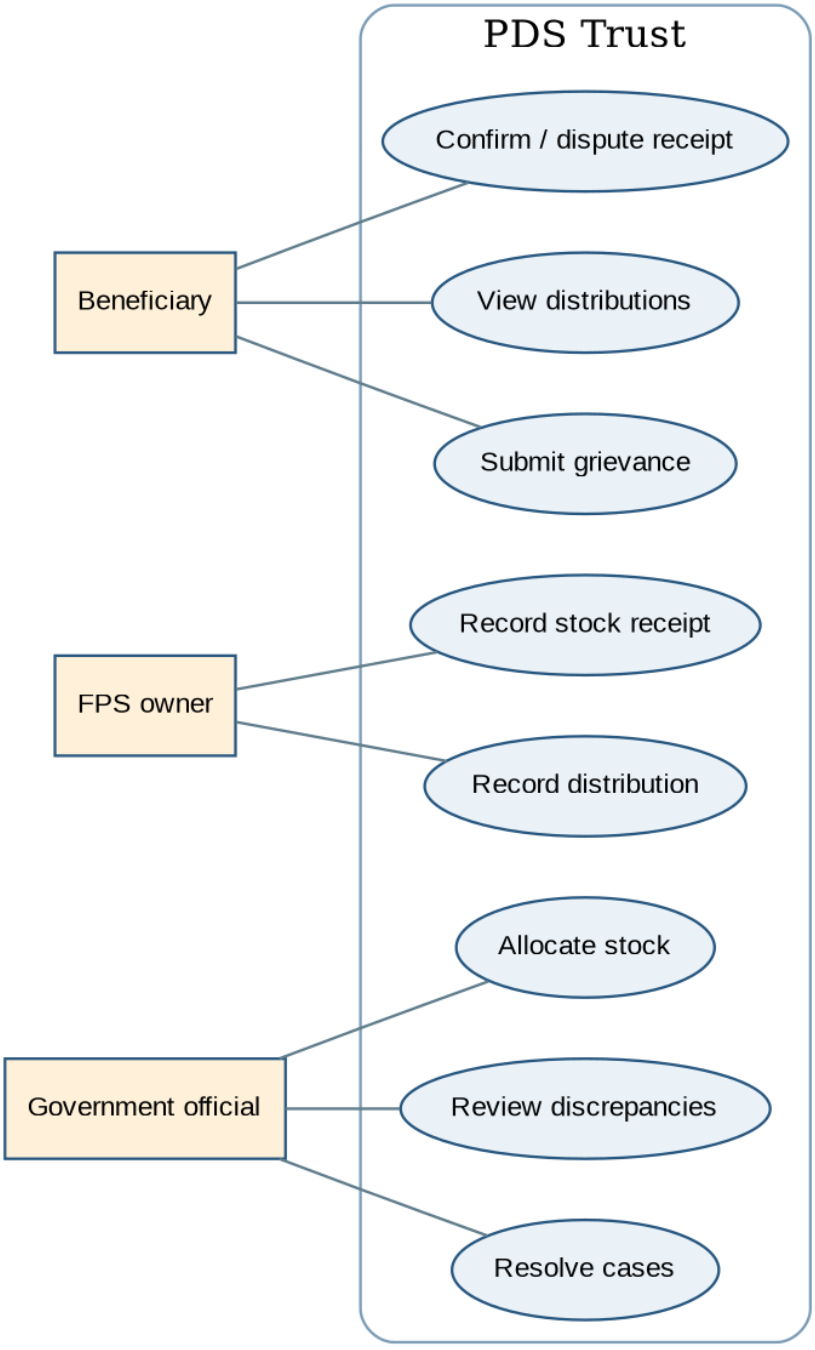


Figure A.1: System architecture.

A.2 UML use case diagram



SMS confirmation/dispute is a second interface to the same use case; it is not a separate user role.

Figure A.2: UML use case diagram.

A.3 DFD Level 0 — context

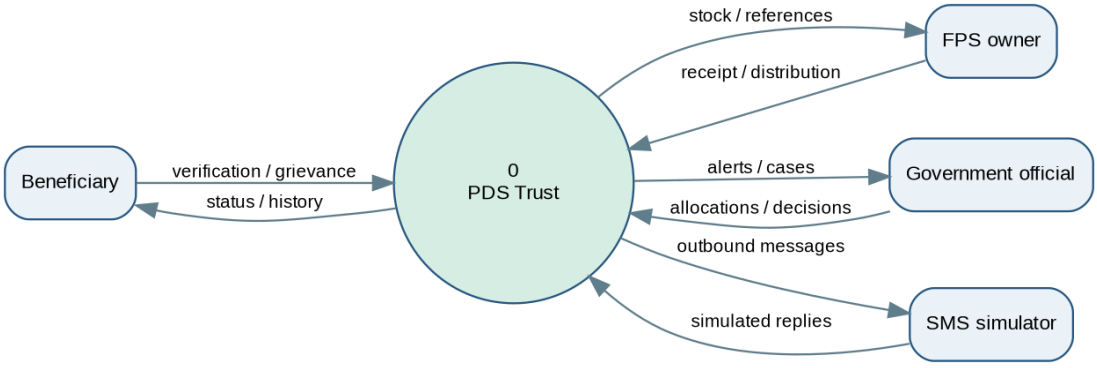
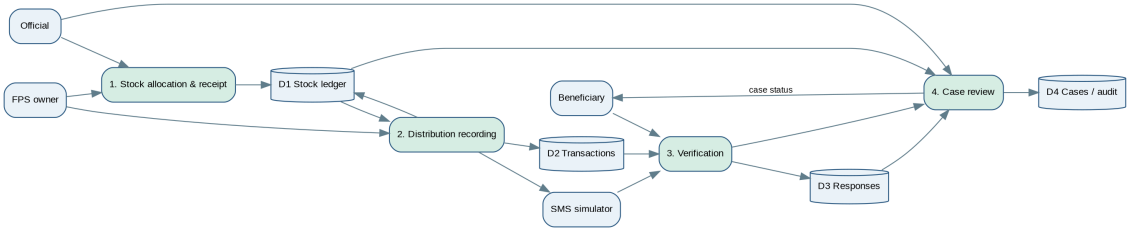


Figure A.3: DFD Level 0 — context.

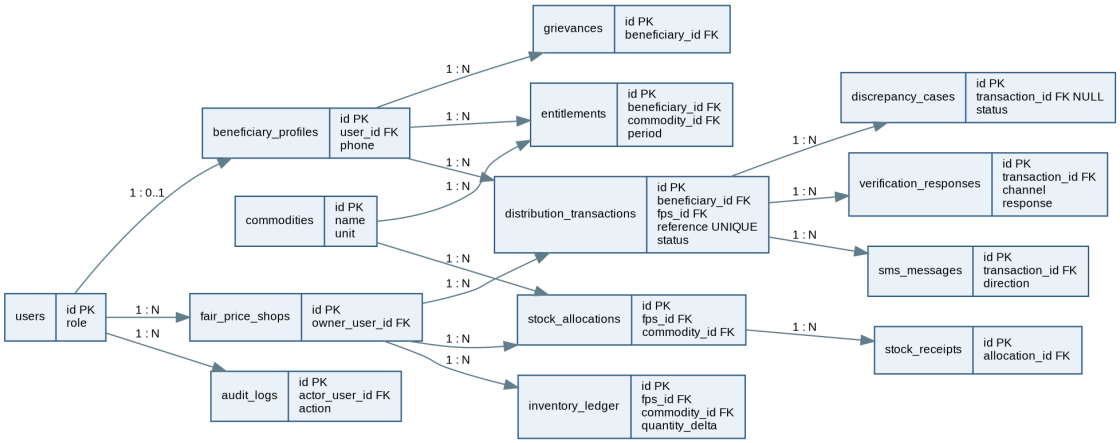
A.4 DFD Level 1 — data flow



Logical DFD; all data stores reside in the prototype database.

Figure A.4: DFD Level 1 — data flow.

A.5 Logical ER diagram



This is a logical model. Commodity links, case sources, and physical FK constraints will be completed and verified against Laravel migrations.

Figure A.5: Logical ER diagram.

A.6 Distribution and verification activity diagram

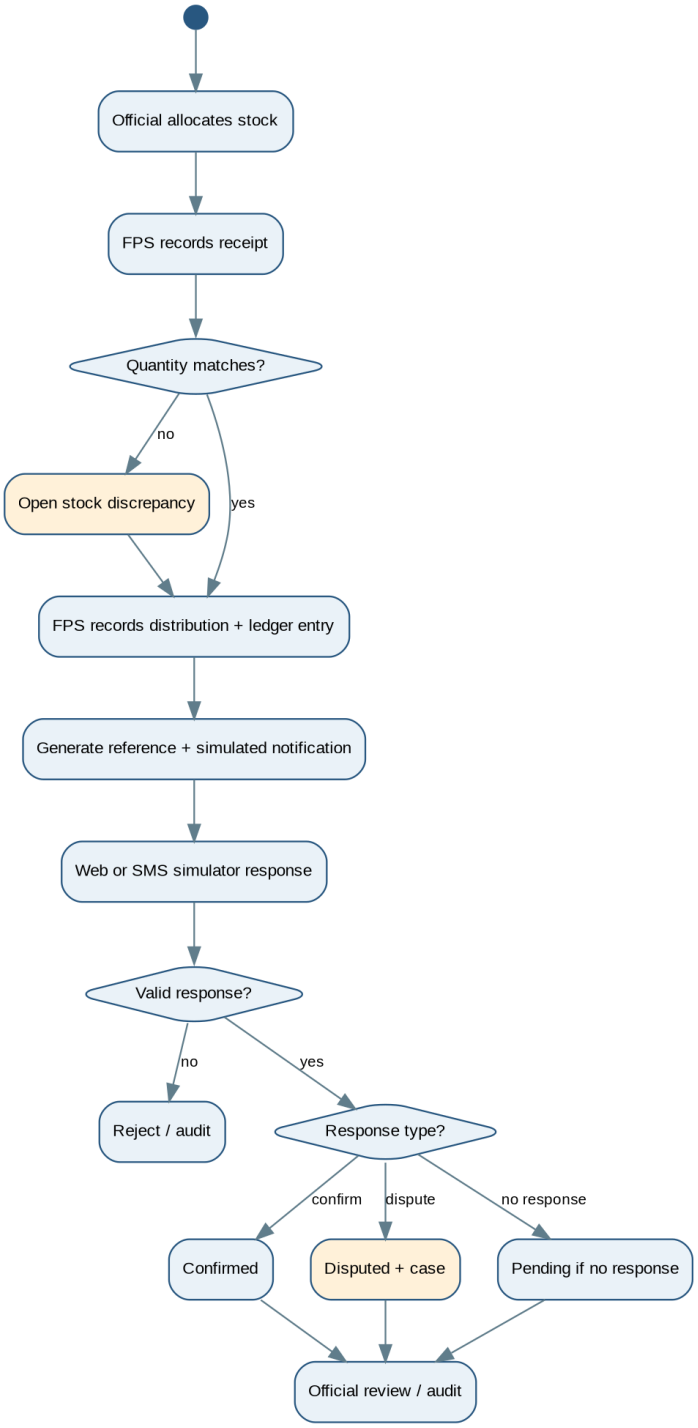
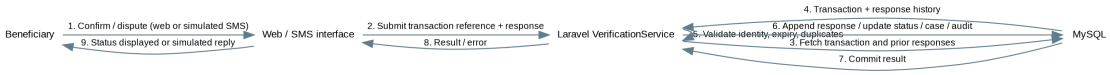


Figure A.6: Distribution and verification activity diagram.

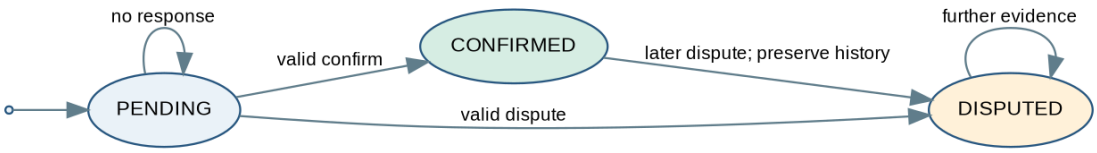
A.7 UML sequence diagram — dual-channel verification



Messages are numbered to show ordering; both interfaces invoke one backend service. The SMS channel is simulated.

Figure A.7: UML sequence diagram — dual-channel verification.

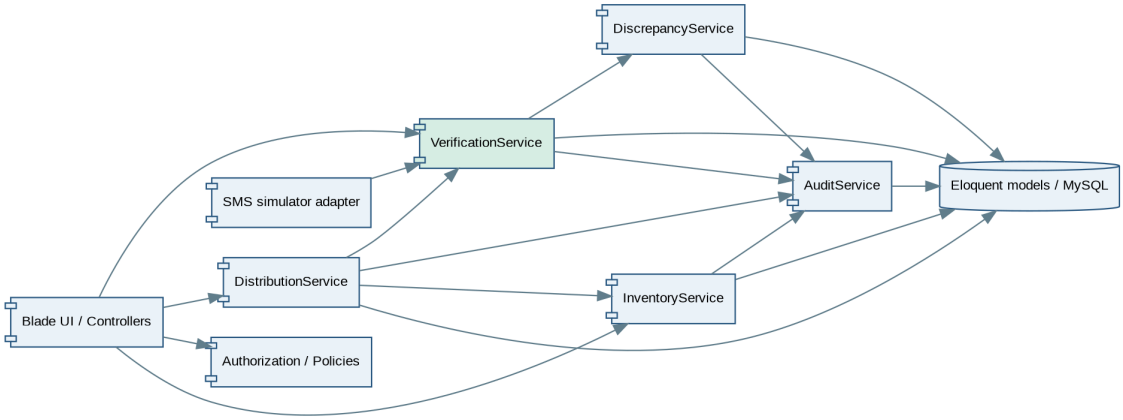
A.8 Verification state machine



No response never implies confirmation; invalid or duplicate replies do not change the verified status.

Figure A.8: Verification state machine.

A.9 UML component diagram — Laravel modules



Component names are proposed Laravel service boundaries, not a claim that these classes are implemented.

Figure A.9: UML component diagram — Laravel modules.

A.10 Deployment diagram and future interfaces

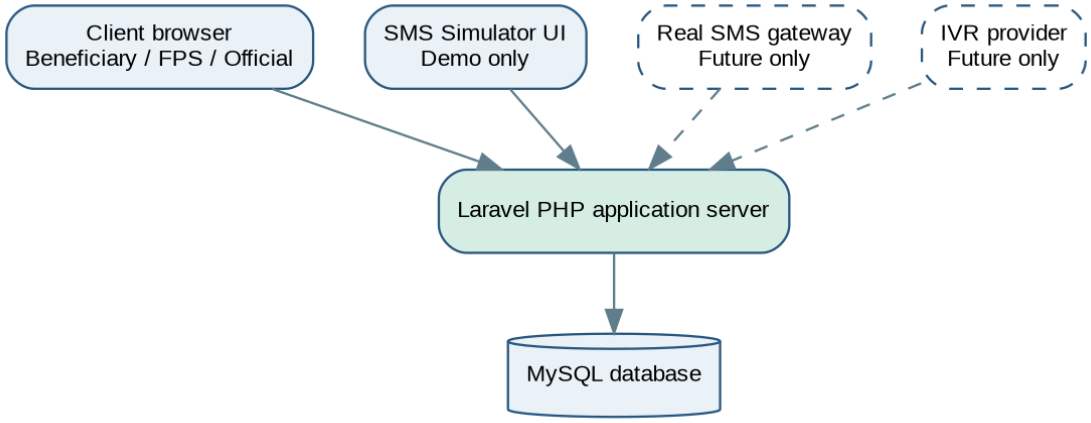


Figure A.10: Deployment diagram and future interfaces.